

Relatório Atividade Prática 3 - INE5413

Alunos:

- Davi Ludvig - 23100473
 - Pedro Henrique Gimenez - 23102766
 - Victória Rodrigues - 23100460
-

Exercício 1

O código implementa o algoritmo de Edmonds-Karp para calcular o fluxo máximo entre dois vértices (s e t) em um grafo dirigido. A abordagem utiliza uma busca em largura (BFS) para encontrar caminhos aumentantes e atualiza os fluxos no grafo iterativamente.

Função `edmonds_karp`

Finalidade: Identificar um caminho aumentante entre o vértice de origem s e o vértice de destino t usando BFS.

Parâmetros:

- `grafo`: Instância de `GrafoDirigido`, representando o grafo direcionado.
- `s`: Vértice de origem.
- `t`: Vértice de destino.

Funcionamento:

1. Reseta os atributos dos vértices (ex.: `acessado`, `anterior`) usando `grafo.limpar_atributos()`.
2. Marca o vértice de origem (s) como acessado e o adiciona à fila Q .
3. Enquanto há vértices na fila:
 - 3.1. Remove o primeiro vértice (u) da fila e verifica seus vizinhos:
 - Caso o vizinho (v) não tenha sido acessado e haja capacidade residual na aresta (u, v), marca-o como acessado e define u como seu vértice anterior.
 - Se v for o vértice de destino (t), reconstrói o caminho aumentante retornando uma lista de rótulos.
4. Se não encontrar um caminho aumentante, retorna `None`.

Função `fluxo_maximo`

Finalidade: Calcula o fluxo máximo no grafo utilizando os caminhos aumentantes obtidos pela função `edmonds_karp`.

Parâmetros:

- `grafo`: Instância de `GrafoDirigido`.
- `s`: Vértice de origem.
- `t`: Vértice de destino.

Funcionamento:

1. Inicializa o fluxo máximo (`fluxo_maximo`) como 0.
 2. Enquanto existem caminhos aumentantes:
 - 2.1. Determina a capacidade mínima (`fluxo_caminho`) ao longo do caminho aumentante.
 - 2.2. Atualiza as capacidades residuais das arestas no caminho, reduzindo o fluxo na direção original e aumentando na direção reversa.
 - 2.3. Adiciona o `fluxo_caminho` ao fluxo máximo.
 3. Retorna o valor total do fluxo máximo.
-

Exercício 2

O algoritmo de Hopcroft-Karp é usado para encontrar o emparelhamento máximo em um grafo bipartido. O emparelhamento máximo é o maior conjunto de arestas que não compartilham vértices.

Função `hopcroft_karp`

Finalidade: Encontrar o emparelhamento máximo em um grafo bipartido usando o algoritmo de Hopcroft-Karp.

Parâmetros:

- `grafo`: Instância de `GrafoBipartido`, representando o grafo bipartido.

Retorno:

- `m`: Inteiro representando o tamanho do emparelhamento máximo.
- `emparelhamento`: Lista de strings com os emparelhamentos no formato `u-v`, onde `u` e `v` são rótulos de vértices emparelhados.

Funcionamento:

1. Inicializa a variável `dist` como um dicionário e `m` (tamanho do emparelhamento) como 0.
2. Enquanto a função `bfs` encontrar caminhos alternantes, realiza uma busca em profundidade (`dfs`) para aumentar o emparelhamento:
 - Se um vértice da partição `X` não está emparelhado (`mate == None`), tenta emparelhá-lo.
 - Incrementa `m` para cada novo emparelhamento encontrado.
3. Após o emparelhamento, cria uma lista com os pares emparelhados no formato `u-v`.
4. Retorna o tamanho do emparelhamento máximo (`m`) e a lista de pares emparelhados.

Função `bfs`

Finalidade: Realizar uma busca em largura (BFS) para determinar as distâncias dos vértices, identificando potenciais caminhos alternantes.

Parâmetros:

- `grafo`: Instância de `GrafoBipartido`.
- `dist`: Dicionário para armazenar as distâncias dos vértices.

Retorno:

- Retorna **True** se existir um caminho alternante, ou **False** caso contrário.

Funcionamento:

1. Inicializa a fila **q** e as distâncias dos vértices em **X**:
 - Vértices não emparelhados (**mate == None**) recebem distância 0 e são adicionados à fila.
 - Vértices emparelhados recebem distância infinita.
2. Define a distância de um vértice **None** (representa o emparelhamento nulo) como infinita.
3. Processa a fila **q**:
 - Para cada vértice **x**, se sua distância for menor que **dist[None]**, verifica seus vizinhos (**y**).
 - Atualiza a distância do par de **y** (**y.mate**) e adiciona-o à fila.
4. Retorna **True** se a distância de **None** não for infinita, indicando a existência de um caminho alternante.

Função **dfs**

Finalidade: Realizar uma busca em profundidade (DFS) para encontrar e aumentar o emparelhamento.

Parâmetros:

- **grafo**: Instância de **GrafoBipartido**.
- **x**: Vértice atual em processamento.
- **dist**: Dicionário com as distâncias calculadas pela BFS.

Retorno:

- Retorna **True** se for possível aumentar o emparelhamento, ou **False** caso contrário.

Funcionamento:

1. Para cada vizinho **y** do vértice **x**:
 - Obtém o vértice **y** e verifica se a distância do par de **y** (**y.mate**) é igual à distância de **x** + 1.
 - Realiza uma chamada recursiva para tentar emparelhar o vértice **y.mate**.
2. Se um emparelhamento for possível:
 - Atualiza **x.mate** como **y** e **y.mate** como **x**.
 - Retorna **True**.
3. Caso contrário, define a distância de **x** como infinita e retorna **False**.

Exercício 3

O algoritmo de Lawler calcula o número cromático de um grafo não dirigido. O número cromático é o número mínimo de cores necessárias para colorir os vértices do grafo de modo que vértices adjacentes não compartilhem a mesma cor.

Função **lawler**

Finalidade: Determinar o número cromático de um grafo não dirigido e fornecer uma coloração válida usando o algoritmo de Lawler.

Parâmetros:

- **grafo**: Instância de **GrafoNaoDirigido**, representando o grafo a ser colorido.

Retorno:

- **numero_cromatico**: Inteiro representando o número mínimo de cores necessárias para colorir o grafo.
- **cores**: Dicionário onde as chaves são os vértices e os valores são as cores atribuídas a cada vértice.

Funcionamento

1. Inicialização:

- **n**: Número de vértices no grafo.
- **vertices**: Lista contendo os rótulos dos vértices.
- **X**: Lista com tamanho (2^n) , onde cada índice representa um subconjunto de vértices, armazenando o número mínimo de cores necessário para esse subconjunto. Inicialmente, todos os valores são infinitos, exceto **X[0]** que é 0 (conjunto vazio).
- **coloracao**: Dicionário que mapeia um subconjunto para a melhor escolha de conjunto independente e o estado restante.

2. Iteração sobre subconjuntos:

- Para cada subconjunto **s** (representado por um inteiro onde o bit (i) indica a presença do vértice (i)):
 - Gera o conjunto **S** de vértices correspondentes.
 - Extrai o subgrafo induzido por **S**.
 - Obtém todos os conjuntos independentes do subgrafo.
 - Para cada conjunto independente **I**:
 - Calcula o subconjunto restante (i) removendo os vértices de **I** de **S**.
 - Atualiza **X[s]** e registra a escolha no dicionário **coloracao** se a nova solução for melhor.

3. Reconstrução da solução:

- Inicia com o subconjunto completo $((1 \ll n) - 1)$ e atribui cores aos vértices dos conjuntos independentes, iterando até cobrir todos os vértices.
- Incrementa a cor atual a cada conjunto independente processado.